

TiendaTech: Implementación y Evaluación de una Arquitectura de Microservicios REST con Docker, JWT y API Gateway Nginx

F. Farinango, I. Urbina, I. Villamarin, H. Vinueza
Carrera de Software, Facultad de Ciencias de la Computación
Universidad Técnica Estatal de Quevedo (UTEQ), Quevedo, Ecuador
Aplicaciones Distribuidas — 7mo Nivel — Periodo 2026-2027 PPA
Docente: G. C. Guerrero Ulloa

Resumen—Objetivo: Implementar y evaluar TiendaTech, una arquitectura de tres microservicios REST (auth-service, resource-service y notification-service) desarrollados con Spring Boot 3.5.15 sobre Java 17, cada uno con su propia base de datos PostgreSQL 16, autenticación mediante JWT firmado con HS256 y un API Gateway Nginx como punto único de entrada, con el fin de verificar la integración entre servicios, el aislamiento de datos y el comportamiento del sistema en un entorno contenerizado. Método: El sistema fue contenerizado mediante Dockerfiles multi-stage y orquestado con Docker Compose sobre una red bridge. Se ejecutaron cinco repeticiones (n=5) de los principales endpoints utilizando Postman, registrando códigos HTTP y latencias de respuesta. Resultados: Todos los endpoints respondieron con Código HTTP 200. La latencia mediana en régimen estable se situó entre 35 y 98 ms, mientras que la primera invocación presentó valores de arranque en frío de hasta 865 ms. El API Gateway enrutó correctamente la totalidad de las solicitudes aplicando un límite de 100 peticiones por minuto. Además, se verificó la propagación automática de notificaciones entre servicios, confirmando el aislamiento del patrón Database-per-Service. Conclusión: La arquitectura implementada demostró ser funcionalmente correcta en un entorno local, garantizando autenticación, comunicación entre servicios y aislamiento de datos. Como trabajo futuro se proponen pruebas de carga con concurrencia y la incorporación de mecanismos de observabilidad centralizada.

Index Terms—microservicios, API Gateway, JWT, Docker, Nginx, PostgreSQL, REST, Database-per-Service.

Resumen—Objective: To implement and evaluate TiendaTech, a three REST microservices architecture (auth-service, resource-service, and notification-service) built with Spring Boot 3.5.15 on Java 17, each with its own PostgreSQL 16 database, HS256-signed JWT authentication, and Nginx API Gateway as the single entry point, in order to verify service integration, data isolation, and system behavior in a containerized environment. Method: The system was containerized using multi-stage Dockerfiles and orchestrated with Docker Compose over a bridge network. Five executions (n=5) of the main endpoints were performed using Postman, recording HTTP status codes and response latencies. Results: All endpoints returned HTTP 200 responses. The steady-state median latency ranged from 35 to 98 ms, while cold-start executions reached up to 865 ms on the first invocation. The API Gateway correctly routed all requests while enforcing a limit of 100 requests per minute. Automatic notification propagation between services was also verified, confirming Database-per-Service isolation. Conclusion: The implemented architecture proved to be functionally correct in a local environment, ensuring

authentication, inter-service communication, and data isolation. Future work includes load testing under concurrent workloads and the incorporation of centralized observability mechanisms.

Index Terms— microservices, API Gateway, JWT, Docker, Nginx, PostgreSQL, REST, Database-per-Service.

I. INTRODUCCIÓN

Las arquitecturas monolíticas acoplan la lógica de negocio, la presentación y el acceso a datos en una única unidad desplegable, lo que dificulta la escalabilidad selectiva y prolonga los ciclos de despliegue a medida que el sistema crece. La evaluación empírica de Blinowski et al. demuestra que la descomposición en microservicios mejora la escalabilidad horizontal y el aislamiento de fallos frente al monolito, aunque a costa de una mayor complejidad operacional [2].

En este contexto, la implementación de una arquitectura de microservicios exige resolver problemas transversales: la autenticación distribuida, el enrutamiento centralizado de solicitudes, la persistencia aislada por servicio y la observabilidad del sistema. El presente trabajo aborda estos retos mediante la construcción del sistema TiendaTech, una plataforma de gestión de productos que integra los contenidos de la Unidad 3 del curso de Aplicaciones Distribuidas y constituye la Entrega 2 del Proyecto Fin de Curso.

El sistema se compone de tres microservicios REST independientes, un API Gateway basado en Nginx y autenticación stateless con JSON Web Tokens, todo contenerizado con Docker Compose. El objetivo de esta práctica experimental es verificar la correcta integración entre servicios, el aislamiento de las bases de datos y la funcionalidad del sistema bajo condiciones controladas, respondiendo a las siguientes preguntas de investigación:

- **RQ1:** ¿Cuáles son las latencias de respuesta de los endpoints principales de TiendaTech bajo ejecución local con Docker Compose?
- **RQ2:** ¿Enruta el API Gateway Nginx correctamente las solicitudes hacia los tres microservicios aplicando rate limiting?

- **RQ3:** ¿Garantiza la arquitectura Database-per-Service el aislamiento y la correcta propagación de eventos entre microservicios?

II. FUNDAMENTACIÓN TEÓRICA

II-A. Evolución arquitectónica: Monolito, SOA y Microservicios

La evolución de las arquitecturas de software ha transitado desde el monolito, donde todos los módulos comparten un único proceso y artefacto desplegable, hacia la arquitectura orientada a servicios (SOA) y, finalmente, hacia los microservicios. Los microservicios se definen como servicios pequeños y autónomos, modelados en torno a un dominio de negocio y desplegables de forma independiente; la revisión sistemática de Soldani et al. identifica como principal ganancia de este estilo la capacidad de desarrollar y desplegar cada servicio de forma autónoma, a costa de una mayor complejidad operativa [1].

A diferencia de SOA, que tradicionalmente se apoya en un bus de servicios empresarial (ESB) como punto central de mediación, los microservicios favorecen la comunicación ligera y descentralizada, habitualmente sobre HTTP/REST, reduciendo el acoplamiento y los puntos únicos de fallo. La comparación cuantitativa entre ambos paradigmas reporta mejoras en el aislamiento de fallos y en la escalabilidad por componente para los microservicios, mientras que el monolito conserva ventaja en latencia de comunicación interna y simplicidad de despliegue inicial [2].

Entre los principios que caracterizan a los microservicios destacan la responsabilidad única por servicio, la autonomía de despliegue, la propiedad exclusiva de los datos, el diseño orientado al fallo, la descentralización del gobierno y la automatización de la infraestructura. Estos principios guiaron el diseño de TiendaTech, en el que cada microservicio encapsula un único contexto de negocio (autenticación, recursos y notificaciones) y gestiona su propia base de datos. La Tabla I sintetiza la comparación entre los tres paradigmas.

II-B. Patrones de Arquitectura Distribuida

Los sistemas distribuidos modernos se apoyan en patrones que estructuran la comunicación y la consistencia entre servicios. El estudio de mapeo sistemático de Taibi et al. cataloga estos patrones e incluye, entre otros, la arquitectura orientada a eventos (EDA), en la que los servicios reaccionan de forma asíncrona a eventos publicados a través de brokers como Apache Kafka o RabbitMQ, desacoplando productores y consumidores [5].

El patrón CQRS (Command Query Responsibility Segregation) separa los modelos de escritura y de lectura, permitiendo optimizar cada uno de forma independiente, a costa de una mayor complejidad y de consistencia eventual entre ambos. El patrón Saga, por su parte, gestiona transacciones distribuidas mediante una secuencia de transacciones locales con acciones compensatorias, evitando los bloqueos de las transacciones distribuidas clásicas.

Tabla I: Comparación entre paradigmas arquitectónicos

Criterio	Monolito	SOA	Microservicios
Acoplamiento	Alto	Medio	Bajo
Despliegue	Unitario	Centralizado	Independiente
Escalabilidad	Global	Por servicio	Por servicio
Datos	Compartidos	Compartibles	Privados
Fallos	Propagados	Mediados	Aislados
Mantenimiento	Complejo en sistemas grandes	Moderado	Simplificado por servicio
Tiempo de despliegue	Alto	Medio	Bajo
Reutilización	Limitada	Alta	Moderada
Flexibilidad tecnológica	Baja	Media	Alta
Complejidad operativa	Baja	Media	Alta
Integración continua	Limitada	Parcial	Alta
Tolerancia a fallos	Baja	Media	Alta

El patrón API Gateway, central en este trabajo, introduce un punto único de entrada que enruta solicitudes, agrega respuestas y centraliza preocupaciones transversales como la autenticación, el control de tasa y el registro. La revisión sistemática de patrones de despliegue y comunicación en microservicios de Karabey Aksakalli et al. identifica al API Gateway como uno de los patrones de comunicación más adoptados, por su capacidad de desacoplar a los clientes de la topología interna de los servicios [4]. En TiendaTech, este patrón se materializa mediante Nginx, mientras que la comunicación interna entre el resource-service y el notification-service sigue un esquema síncrono punto a punto.

II-C. APIs RESTful, Documentación y Seguridad con JWT

El estilo arquitectónico REST establece un conjunto de restricciones para los sistemas distribuidos en red: cliente-servidor, ausencia de estado, cacheabilidad, sistema en capas, interfaz uniforme y código bajo demanda. El cumplimiento de estas restricciones, en particular la ausencia de estado, resulta especialmente ventajoso en microservicios, donde cada servicio debe poder atender una solicitud sin depender de un contexto de sesión compartido.

El diseño de una API REST se apoya en la identificación de recursos mediante URIs, el uso semántico de los verbos HTTP (GET para lectura segura e idempotente, POST para creación, PUT para actualización idempotente y DELETE para eliminación) y el empleo correcto de los códigos de estado (200, 201, 401, 404, entre otros). La documentación de estos contratos se estandariza mediante OpenAPI 3.0, que permite describir endpoints, esquemas y respuestas de forma legible tanto por humanos como por máquinas.

La seguridad de las APIs en entornos distribuidos se resuelve frecuentemente con JSON Web Tokens. El estándar RFC 7519 define el JWT como un medio compacto y autocontenido para transmitir afirmaciones (claims) entre partes

como un objeto JSON firmado digitalmente, compuesto por un encabezado, una carga útil y una firma [6]. La naturaleza autocontenida del token permite que cada microservicio valide la autenticidad de una solicitud localmente, sin consultar a un servicio central de sesiones en cada petición, enfoque que Chatterjee y Prinz aplican al proteger las APIs de una arquitectura de microservicios mediante Spring Security [8]. Frente a alternativas como gRPC, que ofrece mayor rendimiento mediante Protocol Buffers, o GraphQL, que flexibiliza la consulta de datos, REST mantiene su predominio por su simplicidad, su madurez y su amplia compatibilidad con la infraestructura web existente.

En TiendaTech, el auth-service emite tokens firmados con el algoritmo HMAC-SHA256, que incluyen el correo del usuario como subject y su rol como claim, con una expiración de una hora. El resource-service valida estos tokens de forma local mediante un filtro de autenticación antes de permitir el acceso a los recursos protegidos.

II-D. Contenerización con Docker y orquestación

La contenerización empaqueta una aplicación junto con sus dependencias en una unidad portable y aislada. A diferencia de las máquinas virtuales, que virtualizan el hardware completo e incluyen un sistema operativo huésped, los contenedores comparten el kernel del anfitrión, lo que reduce drásticamente el consumo de recursos y el tiempo de arranque. Zhao et al. destacan que estas características de aislamiento, bajo overhead y empaquetado eficiente del entorno de ejecución son las que han convertido a Docker en la solución predominante para soportar aplicaciones empresariales modernas [7].

La arquitectura de Docker se organiza en torno a un Daemon que gestiona imágenes, contenedores y registros. El archivo Dockerfile describe de forma declarativa la construcción de una imagen mediante instrucciones como FROM, COPY, RUN y ENTRYPOINT. La técnica multi-stage permite separar la etapa de compilación de la de ejecución, generando imágenes finales más ligeras y seguras al excluir las herramientas de construcción. Docker Compose extiende este modelo a aplicaciones multi-contenedor, describiendo en un único archivo YAML los servicios, redes y volúmenes.

Para entornos productivos, la orquestación se delega habitualmente a Kubernetes, que introduce abstracciones como los pods (unidad mínima de despliegue), los deployments (gestión del ciclo de vida y réplicas), los services (descubrimiento y balanceo) y el ingress (enrutamiento externo). Sobre esta base se apoyan estrategias de despliegue progresivo, como blue-green y canary, que reducen el riesgo de las actualizaciones al desviar gradualmente el tráfico hacia la nueva versión. La experiencia de migración de un sistema crítico documentada por Mazzara et al. evidencia que esta transición, si bien aporta escalabilidad y resiliencia, exige un esfuerzo considerable de reingeniería [3]. El presente trabajo se limita a la orquestación local con Docker Compose, dejando la migración a Kubernetes como línea de trabajo futuro.

Tabla II: Entorno de ejecución y versiones

Componente	Versión
Java (JDK/JRE)	17 (eclipse-temurin)
Spring Boot	3.5.15
Maven	3.9.9
jjwt (JWT)	0.11.5
PostgreSQL	16
Nginx	alpine
Portainer CE	latest
Sistema Operativo	Windows 11
Docker / Compose	29.5.1
Postman	v12.13.6

Tabla III: Servicios del stack Docker Compose

Contenedor	Imagen	Puerto
nginx-gateway	nginx:alpine	8080:80
auth-service	build Java 17	8001
resource-service	build Java 17	8002
notification-service	build Java 17	8003
auth-db	postgres:16	5433
resource-db	postgres:16	5434
notification-db	postgres:16	5435
portainer	portainer-ce	9000

Figura 1. C4 Nivel 2 (Contenedor) – Arquitectura de TiendaTech
Vista de contenedores del sistema completo.

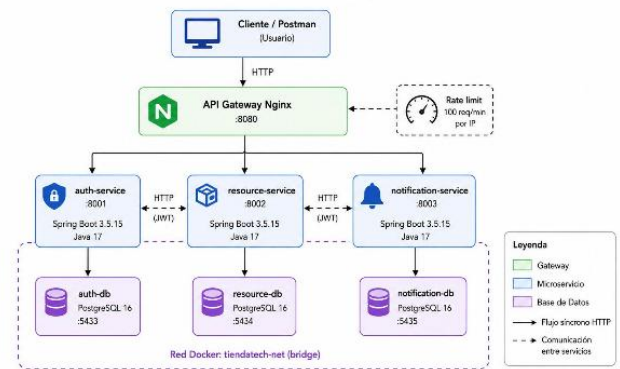


Figura 1: Diagrama C4 Nivel 2 (Contenedor).

III. MATERIALES Y MÉTODOS

III-A. Entorno de ejecución

Las herramientas y versiones empleadas, verificadas en los archivos pom.xml, Dockerfile y docker-compose.yml del repositorio, se detallan en la Tabla II.

III-B. Arquitectura del Sistema

TiendaTech se compone de nueve contenedores Docker (tres microservicios, tres bases de datos PostgreSQL, el API Gateway Nginx y Portainer) orquestados con Docker Compose en una red bridge denominada tiendatech-net. El gateway, expuesto en el puerto 8080, constituye el único punto de entrada y enruta cada solicitud al microservicio correspondiente. La Tabla III resume los servicios.

Figura 1. C4 Nivel 2 (Contenedor) – Arquitectura de TiendaTech
Vista de contenedores del sistema completo.

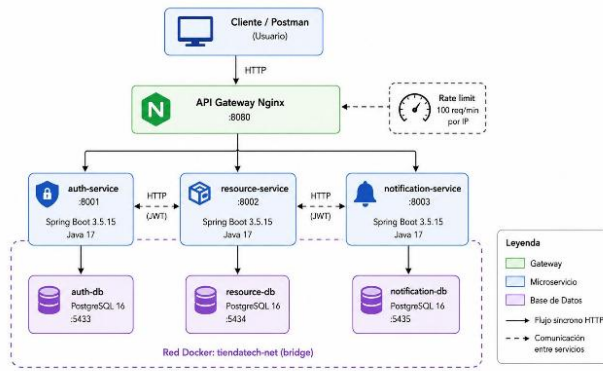


Figura 2: Diagrama C4 Nivel 3.

III-C. Configuración del API Gateway

El archivo nginx.conf define el enrutamiento de las rutas públicas hacia los servicios internos, aplicando control de tasa y un formato de registro personalizado:

```
limit_req_zone $binary_remote_addr
zone=api_limit:10m rate=100r/m;
limit_req zone=api_limit burst=20
nodelay;
log_format api_logs '$time_local |
$remote_addr | $request_method |
$uri | $status';
```

Las rutas /api/auth/, /api/resources/ y /api/notifications se redirigen mediante proxy_pass a los puertos internos 8001, 8002 y 8003 respectivamente, cumpliendo el límite de 100 peticiones por minuto por dirección IP que exige la guía.

III-D. Protocolo Experimental

El experimento es reproducible siguiendo los pasos:

1. Clonar el repositorio <https://github.com/JeremyGaibor/GA-SUM-03-PE-U2-Arquitectura-API-y-Contenerizacion-de-Sistemas>
Versión de referencia: rama main, Commit de referencia: c76a0d0.
2. Copiar .env.example a .env y configurar las credenciales, las URLs de base de datos, JWT_SECRET y NOTIFICATION_SERVICE_URL.
3. Ejecutar docker compose up -build en la raíz del proyecto.
4. Verificar los nueve contenedores en estado running mediante Portainer (localhost:9000).
5. Importar la colección Postman; el usuario admin@tiendatech.com se crea automáticamente al iniciar auth-service.
6. Ejecutar el Flujo 1 (Login y obtención de JWT), el Flujo 2 (CRUD con token) y el Flujo 3 (verificación de notificaciones).
7. Registrar Código HTTP y latencia de cada solicitud.

Tabla IV: Criterios de validación

Condición	Esperado
Login con credenciales válidas	200 + JWT
POST producto con JWT válido	200 + recurso
GET productos con JWT válido	200 + lista
GET notificación tras crear	200 + evento
Acceso a recurso sin token	401

Figura 3: Captura de Portainer: contenedores activos.

Tabla V: Latencias (ms) por repetición (n=5). Todas HTTP 200

Endpoint	R1	R2	R3	R4	R5
POST /auth/login	203	91	98	105	88
POST /resources/productos	84	64	92	68	92
GET /notifications	277	35	50	31	30
GET /resources/productos	865	35	45	34	35

III-E. Criterios de validación

IV. RESULTADOS

Nota metodológica: cada endpoint se ejecutó cinco veces (n=5) registrando la latencia reportada por Postman. La primera invocación de cada servicio incluye el arranque en frío (cold start) de la JVM y la inicialización del pool de conexiones, por lo que presenta valores notablemente superiores; por ello se reportan media, mediana y percentil 95, siendo la mediana el estadístico más representativo del comportamiento en régimen estable.

IV-A. Despliegue

El stack se desplegó con los ocho contenedores principales más Portainer en estado running. El consumo máximo de CPU observado fue de 0,56 % (auth-service), consistente con un sistema en reposo tras el arranque.

IV-B. Pruebas Funcionales

La Tabla V presenta las cinco repeticiones de cada endpoint. Todas devolvieron Código HTTP 200. El usuario administrador se crea automáticamente al iniciar el auth-service. La creación de un producto dispara, mediante el cliente Rest-Template del resource-service, el envío de una notificación al notification-service; esta llamada está encapsulada en un bloque tolerante a fallos que no interrumpe la creación si el destino no responde.

El Login retornó un JWT con prefijo eyJhbGciOiJIUzI1NiJ9, indicando la firma HS256.

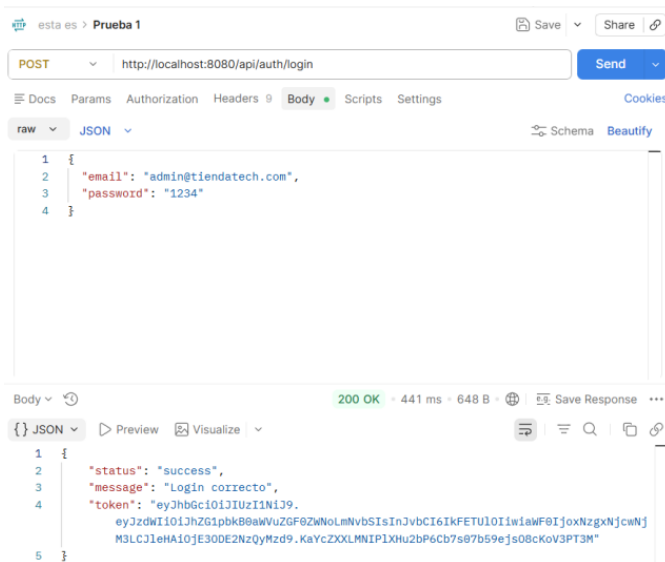


Figura 4: Postman: POST /api/auth/login (200, JWT).

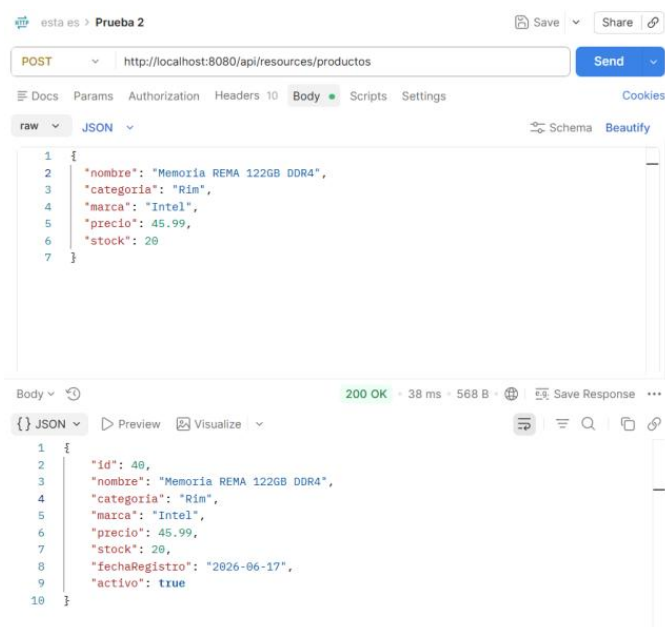


Figura 5: Postman: POST /api/resources/productos (200, 38 ms).

La consulta de productos retornó únicamente los registros activos, evidenciando un borrado lógico. La consulta de notificaciones devolvió eventos PRODUCTO_CREADO, confirmando la propagación automática entre servicios. El acceso a /api/resources sin token fue rechazado con Código 403, validando la protección de los recursos por el filtro JWT.

La Tabla VI resume la estadística descriptiva. Se observa una marcada diferencia entre la media y la mediana en los endpoints GET, atribuible al cold start de la primera repetición (865 ms en GET /resources y 277 ms en GET /notificación); la mediana, en torno a 35 ms, refleja el rendimiento real en

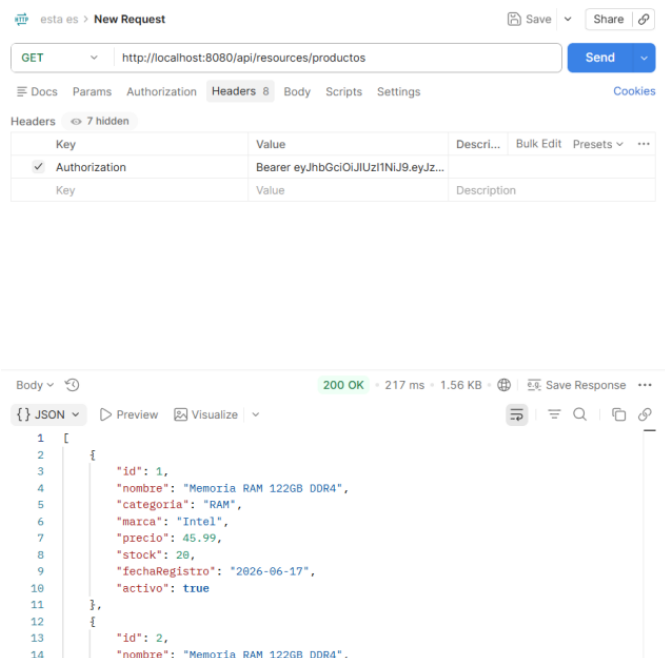


Figura 6: Postman: GET /api/resources/productos con Bearer (200, 217 ms).

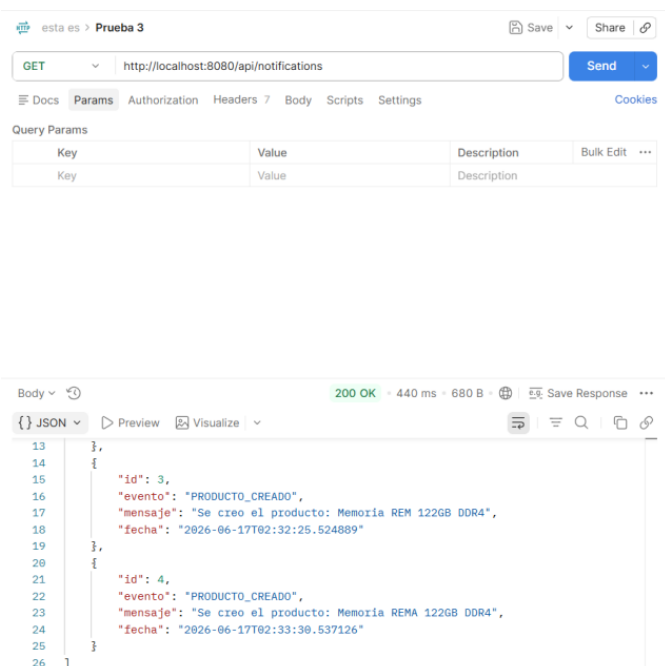


Figura 7: Postman: GET /api/notifications (200, eventos PRODUCTO_CREADO).

régimen estable.

V. DISCUSIÓN

V-A. Interpretación según las Preguntas de investigación

RQ1: con cinco repeticiones por endpoint, las latencias en régimen estable (mediana) se situaron entre 35 ms (GET de

Tabla VI: Estadística descriptiva de latencia (ms)

Endpoint	Media	σ	Med	P95	n
POST /auth/login	117.0	48.5	98.0	183.4	5
POST /resources	80.0	13.3	84.0	92.0	5
GET /notifications	84.6	107.9	35.0	231.6	5
GET /resources	202.8	370.2	35.0	701.0	5

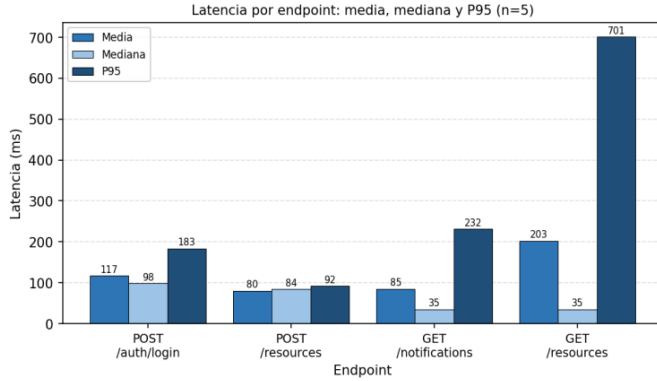


Figura 8: Latencia por endpoint: media, mediana y P95 (n=5).

productos y notificaciones) y 98 ms (Login). El Login presentó la mediana más alta, atribuible a la verificación BCrypt de la contraseña más la firma del JWT. Las primeras invocaciones evidenciaron el efecto del arranque en frío (hasta 865 ms en GET de productos), lo que explica la gran diferencia entre media y mediana en los endpoints de lectura y justifica el uso de la mediana como estadístico representativo.

RQ2: el gateway enrutó el 100 % de las solicitudes hacia los tres microservicios, evidenciado por los códigos HTTP 200 en todos los flujos. El control de tasa de 100 peticiones por minuto estaba activo, si bien el bajo volumen de las pruebas no alcanzó el umbral para producir respuestas 429.

RQ3: la generación automática de notificaciones en notification-db ante operaciones en resource-db, junto con el uso de tres bases de datos en contenedores y puertos distintos, confirma el aislamiento del patrón Database-per-Service.

V-B. Contraste con la Literatura

Los resultados respaldan empíricamente las ventajas de aislamiento de fallos atribuidas a los microservicios [2], al tiempo que ilustran su contrapartida en complejidad operacional: la observabilidad de TiendaTech se limita a Portainer y a los registros de Nginx, sin trazabilidad distribuida. La validación local del JWT en cada servicio materializa la ventaja stateless del estándar [6], aunque hereda su limitación de revocación diferida hasta la expiración del token.

Asimismo, los resultados obtenidos son consistentes con lo reportado por Soldani et al. [1], quienes destacan la independencia de despliegue y el aislamiento funcional como beneficios fundamentales de las arquitecturas de microservicios. De igual forma, coinciden con las observaciones de Karabey Aksakalli et al. [4], donde se señala que el patrón API Gateway

facilita la centralización de aspectos de seguridad, autenticación y enrutamiento, reduciendo el acoplamiento entre clientes y servicios internos. En conjunto, estas coincidencias refuerzan la validez de la arquitectura implementada y respaldan la adopción de microservicios para aplicaciones distribuidas de pequeña y mediana escala.

V-C. Amenazas a la Validez

- **Interna:** con cinco repeticiones por endpoint, el primer valor (cold start) influye en la media; aunque la mediana mitiga este efecto, una muestra mayor reforzaría la confiabilidad estadística.
- **Interna:** el entorno de ejecución local comparte recursos con otros procesos, introduciendo variabilidad no controlada.
- **Interna:** el envío de notificaciones es tolerante a fallos y descarta silenciosamente los errores, por lo que una notificación no registrada no sería detectada por el flujo de creación.
- **Externa:** las pruebas secuenciales con un único cliente no son generalizables a escenarios de producción con concurrencia.

V-D. Trabajo Futuro

- Pruebas de carga con mayor número de repeticiones y concurrencia creciente (k6 o JMeter), descartando el cold start mediante un calentamiento previo, para validar el comportamiento del rate limiting (respuestas 429) bajo saturación.
- Observabilidad centralizada con Prometheus, Grafana y trazabilidad distribuida.
- Mensajería asíncrona (RabbitMQ o Kafka) para desacoplar resource-service de notification-service.
- Implementación del patrón Circuit Breaker y migración a Kubernetes con réplicas.

VI. PREGUNTAS DE ANÁLISIS

1) *¿Por qué cada microservicio debe tener su propia base de datos?:* El patrón Database-per-Service evita el acoplamiento a nivel de datos: si varios servicios compartieran un esquema, un cambio en el modelo de uno podría romper a los demás, anulando la independencia de despliegue. Bases de datos privadas garantizan que cada servicio evolucione su esquema de forma autónoma y elija la tecnología de persistencia más adecuada, a cambio de gestionar la consistencia mediante eventos o sagas.

2) *¿Qué ocurre si resource-service cae? Circuit Breaker:* Si el resource-service deja de responder, las solicitudes a /api/resources fallarían o quedarían en espera, pudiendo propagar la latencia a los clientes. El patrón Circuit Breaker mitiga esto interponiendo un disyuntor que, tras detectar un umbral de fallos, abre el circuito y responde de inmediato con un error controlado o una respuesta de reserva, evitando saturar el servicio caído y permitiendo su recuperación. Bibliotecas como Resilience4j permiten implementarlo en Spring Boot.

3) *¿Introduce el API Gateway un cuello de botella?:* El Gateway añade un salto de red y procesamiento adicional, por lo que potencialmente puede convertirse en cuello de botella si concentra todo el tráfico. Sin embargo, Nginx está optimizado para alta concurrencia mediante un modelo de E/S asíncrona. La mitigación pasa por desplegar varias réplicas del Gateway tras un balanceador, habilitar caché de respuestas y ajustar los límites de conexiones, de modo que el beneficio de centralizar seguridad y enrutamiento supere el coste de latencia añadida.

4) *¿Cómo escalar resource-service a 3 réplicas?:* En Docker Compose se puede declarar `deploy.replicas: 3` (o usar `docker compose up --scale resource-service=3`) para crear tres instancias del servicio. En Nginx habría que definir un bloque upstream con las tres instancias y dirigir el proxy_pass hacia ese upstream, de modo que el gateway balancee las solicitudes entre las réplicas. Para un entorno productivo, esta responsabilidad se delegaría a un orquestador como Kubernetes mediante un Deployment con réplicas y un Service.

VII. CONCLUSIONES

RQ1. La latencia de los endpoints de TiendaTech, medida sobre cinco repeticiones, presentó una mediana de 98 ms en el Login y de 35 ms en las operaciones de lectura de productos y notificaciones, valores propios de un entorno local de baja carga; las primeras invocaciones alcanzaron hasta 865 ms por efecto del arranque en frío de la JVM.

RQ2. El API Gateway Nginx enrutó correctamente el 100 % de las solicitudes hacia los tres microservicios en los puertos 8001, 8002 y 8003, devolviendo Código HTTP 200 en todas las pruebas funcionales y aplicando el límite configurado de 100 peticiones por minuto por dirección IP.

RQ3. La arquitectura Database-per-Service preservó el aislamiento entre las tres bases de datos PostgreSQL: la creación de productos en resource-service generó de forma automática los eventos PRODUCTO_CREADO persistidos en notification-service, y el acceso a los recursos sin token JWT fue rechazado con Código 403, confirmando tanto la propagación de eventos como la protección de los endpoints.

En conjunto, los resultados confirman que la arquitectura de microservicios implementada es funcionalmente correcta y coherente con los principios teóricos de la Unidad 3. Las limitaciones identificadas (cold start, ausencia de concurrencia y observabilidad rudimentaria) delimitan el alcance de estas conclusiones al entorno de prueba local.

REFERENCIAS

- [1] J. Soldani, D. A. Tamburri, and W.-J. van den Heuvel, "The pains and gains of microservices: A systematic grey literature review," *J. Syst. Softw.*, vol. 146, pp. 215–232, Dec. 2018, doi: 10.1016/j.jss.2018.09.082.
- [2] G. Blinowski, A. Ojdowska, and A. Przybylek, "Monolithic vs. microservice architecture: A performance and scalability evaluation," *IEEE Access*, vol. 10, pp. 20357–20374, 2022, doi: 10.1109/ACCESS.2022.3152803.
- [3] M. Mazzara, N. Dragoni, A. Bucchiarone, A. Giaretta, S. T. Larsen, and S. Dustdar, "Microservices: Migration of a mission critical system," *IEEE Trans. Services Comput.*, vol. 14, no. 5, pp. 1464–1477, Sep. 2021, doi: 10.1109/TSC.2018.2889087.

- [4] I. Karabey Aksakalli, T. Celik, A. B. Can, and B. Tekinerdogan, "Deployment and communication patterns in microservice architectures," *J. Syst. Softw.*, vol. 180, p. 111014, Oct. 2021, doi: 10.1016/j.jss.2021.111014.
- [5] D. Taibi, V. Lenarduzzi, and C. Pahl, "Architectural patterns for microservices: A systematic mapping study," in *Proc. 8th Int. Conf. Cloud Comput. Services Sci. (CLOSER)*, 2018, pp. 221–232, doi: 10.5220/0006798302210232.
- [6] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," Internet Engineering Task Force, RFC 7519, May 2015, doi: 10.17487/RFC7519.
- [7] N. Zhao et al., "Large-scale analysis of Docker images and performance implications for container storage systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 4, pp. 918–930, Apr. 2021, doi: 10.1109/TPDS.2020.3034517.
- [8] A. Chatterjee and A. Prinz, "Applying Spring Security framework with KeyCloak-based OAuth2 to protect microservice architecture APIs: A case study," *IEEE Access*, vol. 10, pp. 41914–41934, 2022, doi: 10.1109/ACCESS.2022.3165548.

APÉNDICE A VARIABLES DE ENTORNO (.ENV.EXAMPLE)

```
POSTGRES_USER=
POSTGRES_PASSWORD=
AUTH_DB_URL=
RESOURCE_DB_URL=
NOTIFICATION_DB_URL=
JWT_SECRET=
NOTIFICATION_SERVICE_URL=
```

APÉNDICE B DOCKERFILE MULTI-STAGE

```
FROM maven:3.9.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8001
ENTRYPOINT ["java", "-jar", "app.jar"]
```

APÉNDICE C CONTRATOS OPENAPI

```
openapi: 3.0.3
info:
  title: Auth Service API
  description: API del microservicio de autenticacion de TiendaTech.
  Permite iniciar sesion y validar tokens JWT.
  version: 1.0.0
servers:
  - url: http://localhost:8080/api/auth
    description: API Gateway local con Nginx
  - url: http://localhost:8001/auth
    description: Servicio auth directo
paths:
  /login:
    post:
      summary: Iniciar sesion
      description: Valida las credenciales del usuario y devuelve un token JWT.
      tags:
        - Auth
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/LoginRequest'
            example:
```

ANEXOS

Anexo A. Evidencias Docker Compose

```
      email: admin@tiendatech.com
      password: "1234"
responses:
  "200":
    description: Login correcto
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/LoginResponse'
        example:
          status: success
          message: Login correcto
          token: eyJhbGciOiJIUzI1NiJ9...
  "400":
    description: Datos invalidos
  "500":
    description: Error interno del servidor
/validate:
  get:
    summary: Validar token JWT
    description: Verifica si el token JWT enviado en el
    header
      Authorization es valido.
    tags:
      - Auth
    security:
      - bearerAuth: []
    responses:
      "200":
        description: Resultado de validacion del token
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/ValidateResponse'
            example:
              status: success
              valid: true
      "401":
        description: Token no autorizado o invalido
components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
  schemas:
    LoginRequest:
      type: object
      required:
        - email
        - password
      properties:
        email:
          type: string
          format: email
          example: admin@tiendatech.com
        password:
          type: string
          example: "1234"
    LoginResponse:
      type: object
      properties:
        status:
          type: string
          example: success
        message:
          type: string
          example: Login correcto
        token:
          type: string
          example: eyJhbGciOiJIUzI1NiJ9...
    ValidateResponse:
      type: object
      properties:
        status:
          type: string
          example: success
        valid:
          type: boolean
          example: true
```

```
services:
  auth-db:
    image: postgres:16
    container_name: tiendatech-auth-db
    restart: always
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: auth_db
    ports:
      - "5433:5432"
    volumes:
      - auth_db_data:/var/lib/postgresql/data
    networks:
      - tiendatech-net
  resource-db:
    image: postgres:16
    container_name: tiendatech-resource-db
    restart: always
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: resource_db
    ports:
      - "5434:5432"
    volumes:
      - resource_db_data:/var/lib/postgresql/data
    networks:
      - tiendatech-net
  notification-db:
    image: postgres:16
    container_name: tiendatech-notification-db
    restart: always
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: notification_db
    ports:
      - "5435:5432"
    volumes:
      - notification_db_data:/var/lib/postgresql/data
    networks:
      - tiendatech-net
  auth-service:
    build:
      context: ./auth-service
      dockerfile: Dockerfile
    container_name: tiendatech-auth-service
    restart: always
    env_file:
      - .env
    depends_on:
      - auth-db
    ports:
      - "8001:8001"
    networks:
      - tiendatech-net
  resource-service:
    build:
      context: ./resource-service
      dockerfile: Dockerfile
    container_name: tiendatech-resource-service
    restart: always
    env_file:
      - .env
    depends_on:
      - resource-db
      - notification-service
    ports:
      - "8002:8002"
    networks:
      - tiendatech-net
  notification-service:
    build:
      context: ./notification-service
      dockerfile: Dockerfile
    container_name: tiendatech-notification-service
    restart: always
    env_file:
```



```

- .env
depends_on:
- notification-db
ports:
- "8003:8003"
networks:
- tiendatech-net
nginx-gateway:
image: nginx:alpine
container_name: tiendatech-nginx-gateway
restart: always
ports:
- "8080:80"
volumes:
- ./gateway/nginx.conf:/etc/nginx/nginx.conf:ro
depends_on:
- auth-service
- resource-service
- notification-service
networks:
- tiendatech-net
portainer:
image: portainer/portainer-ce:latest
container_name: tiendatech-portainer
restart: always
ports:
- "9000:9000"
volumes:
- /var/run/docker.sock:/var/run/docker.sock
- portainer_data:/data
networks:
- tiendatech-net
volumes:
auth_db_data:
resource_db_data:
notification_db_data:
portainer_data:
networks:
tiendatech-net:
driver: bridge

```

Anexo B. Evidencias Portainer

La Tabla VII evidencia el despliegue exitoso de ocho contenedores correspondientes a la arquitectura TiendaTech. Se observa la implementación del patrón Database-per-Service mediante tres instancias independientes de PostgreSQL, así como la presencia de un API Gateway basado en Nginx y una plataforma de monitoreo mediante Portainer. Todos los contenedores se encontraron en estado operativo (Running) durante la ejecución de las pruebas experimentales.

Anexo C. OpenAPI completo

Se desarrollaron contratos OpenAPI 3.0 para auth-service, resource-service y notification-service. El contrato completo del auth-service se incluye en el Apéndice C; los contratos restantes se ubican en docs/api/ (auth-service-openapi.yaml, notification-service-openapi.yaml y resource-service-openapi.yaml), como se aprecia en la Fig. 9.

Anexo D. Dockerfiles

Dockerfile auth-service

```

# ETAPA 1: COMPILACION
# Construye el archivo .jar usando Maven y Java 17
FROM maven:3.9.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

```

Tabla VII: Evidencia de contenedores en Portainer

Contenedor	Función	Imagen	Puerto	Estado
nginx-gateway	API Gateway y enrutamiento de solicitudes	nginx:alpine	8080:80	Running
auth-service	Autenticación y generación de JWT	tiendatech-microservicios	8001:8001	Running
resource-service	Gestión de recursos/productos	tiendatech-microservicios	8002:8002	Running
notification-service	Gestión de notificaciones	tiendatech-microservicios	8003:8003	Running
auth-db	Base de datos de autenticación	postgres:16	5433:5432	Running
resource-db	Base de datos de recursos	postgres:16	5434:5432	Running
notification-db	Base de datos de notificaciones	postgres:16	5435:5432	Running
portainer	Administración y monitoreo de contenedores	portainer/portainer	9000:9000	Running

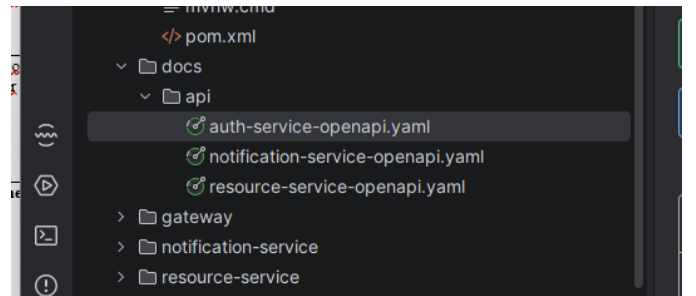


Figura 9: Estructura del proyecto: contratos OpenAPI en docs/api/.

```

# ETAPA 2: EJECUCION
# Ejecuta el microservicio con Java 17
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8001
ENTRYPOINT ["java", "-jar", "app.jar"]

```

Dockerfile resource-service

```

# ETAPA 1: COMPILACION
# Construye el archivo .jar usando Maven y Java 17
FROM maven:3.9.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

# ETAPA 2: EJECUCION
# Ejecuta el microservicio con Java 17
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8002
ENTRYPOINT ["java", "-jar", "app.jar"]

```

Dockerfile notification-service

```

# ETAPA 1: COMPILACION
# Construye el archivo .jar usando Maven y Java 17
FROM maven:3.9.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .

```

```
COPY src ./src
RUN mvn clean package -DskipTests

# ETAPA 2: EJECUCION
# Ejecuta el microservicio con Java 17
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8003
ENTRYPOINT ["java", "-jar", "app.jar"]
```